

突破语义鸿沟：基于 LLM 技术的软件漏洞分析与挖掘技术发展现状调研报告

摘要

随着云原生架构的普及与软件供应链复杂度的指数级攀升，现代软件系统的安全性面临前所未有的挑战。传统的自动化漏洞挖掘技术主要依赖两大支柱：基于规则的静态分析（SAST）与基于覆盖率反馈的动态模糊测试（Fuzzing）。然而，前者因缺乏语义理解而深陷高误报率的泥潭，后者在面对深层业务逻辑和高度结构化输入时往往触及瓶颈。近年来，以大语言模型（LLM）为代表的生成式人工智能技术，凭借其卓越的代码语义理解与逻辑推理能力，为软件安全领域带来了范式级的转移。本文深入调研了 2023 至 2025 年间，基于 LLM 的漏洞挖掘技术从早期的“静态代码分类”向“生成式模糊测试”与“自主攻防智能体（Agent）”协同演进的完整历程。报告重点剖析了 LLM 在通用模糊测试种子生成（如 Fuzz4All 框架）、交互式自动化渗透测试（如 PentestGPT）以及多语言漏洞基准测试中的核心机制与实际表现。研究表明，尽管 LLM 在理解代码语义与生成攻击载荷上超越了上一代深度学习模型，但在处理跨文件（Repository-level）上下文依赖、消除模型“幻觉”以及应对数据污染带来的评估偏差方面，仍面临严峻的工程挑战。未来的技术趋势将不可避免地指向神经符号 AI 与多智能体（Multi-Agent）协同攻防的深度融合。

一、引言：语义鸿沟下的技术困境与破局

软件漏洞挖掘一直是网络攻防领域的“皇冠明珠”。在过去的二十年里，该领域经历了从依赖专家经验的手工审计到自动化工具辅助的演变。然而，面对当今动辄数百万行代码的大型软件项目，传统技术手段逐渐显露出“语义鸿沟”的局限性。以 Coverity、Checkmarx 为代表的商业化静态分析工具，其核

心仍基于抽象语法树（AST）和控制流图（CFG）的模式匹配。这种方法虽然能有效捕捉缓冲区溢出或空指针解引用等语法层面的错误，但由于缺乏对代码“意图”的真正理解，难以识别业务逻辑绕过、权限校验缺失等深层缺陷，且往往产生海量误报，导致安全人员陷入“告警疲劳”。另一方面，以 AFL++ 为代表的模糊测试工具虽然在发现内存破坏类漏洞上战功卓著，但在面对编译器、解释器或复杂网络协议等高度结构化的输入时，随机变异策略生成的测试用例绝大多数会被程序的语法校验层拒绝，导致计算资源大量浪费在浅层路径上，难以触达核心逻辑。

2023 年以来，随着 GPT-4、Claude 3.5 Sonnet、CodeLlama 等大语言模型的爆发，安全研究界开始探索利用 LLM 的通用推理能力来填补这一鸿沟。与早期的基于 BERT 的代码模型（如 CodeBERT）仅能进行简单的分类预测不同，现代生成式 LLM 具备了两个核心优势：一是深层语义理解，能够“读懂”变量命名、注释以及函数间的隐式依赖关系；二是强大的生成与规划能力，不仅能指出漏洞，还能编写通过编译的攻击载荷（Payload），甚至自主编写 Fuzzer 的驱动代码（Harness）。进入 2025 年，技术重心已从单点的“大模型检测”转向了体系化的“Agent 驱动攻防”，旨在构建能够像人类专家一样进行多步推理、工具调用和环境交互的智能系统。

二、核心技术范式演进：从辅助识别到自主攻防

基于 LLM 的漏洞挖掘技术并非一蹴而就，其技术路线在过去三年中经历了从判别式到生成式，再到交互式的三次跨越，每一次跨越都极大地拓展了自动化挖掘的边界。

1. 上下文学习重塑静态检测

早期的尝试（约 2022-2023 年）主要集中在“预训练与微调”范式。研究人员试图将小参数量的 LLM 视为一个二分类器，通过在 SARD、BigVul 等数据集上进行全量微调，训练其判断代码片段是否“Vulnerable”。然而，这种方法高度依赖标注数据的质量，且容易导致模型过拟合，难以应对现实世界中千变万化的漏洞形态。随着模型参数量的提升，主流技术迅速转向了上下文学习（In-Context Learning）。通过在 Prompt 中嵌入少样本（Few-shot）漏洞示

例，或者利用思维链（Chain-of-Thought, CoT）引导模型分步骤分析数据流向（Source-to-Sink），模型能够在不更新权重的情况下，利用其在海量代码库中习得的通用知识，推理出未见过的漏洞模式。这种能力的跃升，使得零样本（Zero-shot）漏洞检测成为可能，尤其是在发现传统工具难以识别的逻辑漏洞（如加密算法误用、IDOR）方面，LLM 展现出了类似人类专家的直觉。

2. 生成式模糊测试打破语法壁垒

这是 LLM 与传统安全技术结合最成功的领域之一。传统 Fuzzer 的痛点在于“不懂语法”，而 LLM 天生具备强大的语法生成能力。Xia 等人（ICSE 2024）提出的 Fuzz4All 框架标志着这一技术的成熟[1]。该工具不再依赖特定语言的语法模板，而是利用 LLM 根据简短的输入描述或 API 文档，自动生成多样的、覆盖率极高的模糊测试输入。更为关键的是，LLM 开始扮演“智能变异器”的角色。当传统 Fuzzer 卡在某个复杂的校验条件（如 Magic Number 校验或校验和计算）时，LLM 可以通过分析导致路径阻塞的代码片段，反向生成满足特定约束的输入，从而引导 Fuzzer 突破瓶颈，深入探索深层逻辑。这种“大模型大脑 + Fuzzer 手脚”的混合架构，大幅提升了漏洞挖掘的深度与广度。

3. 自主智能体实现交互式渗透

进入 2024-2025 年，技术的前沿已拓展至“自主智能体（Autonomous Agents）”。与单次问答不同，安全 Agent 被设计为具备长期记忆、工具使用能力和自我规划能力的系统。在 Web 渗透测试等需要多步交互的场景中，Agent 采用了 ReAct（Reasoning and Acting）框架，模拟人类渗透专家的思维过程。Deng 等人在 USENIX Security 2024 上发布的 PentestGPT 是该领域的里程碑之作[2]。该系统通过推理、生成、解析三个模块的闭环交互，能够自主完成“观察网页结构 -> 制定攻击计划 -> 生成 SQLMap 命令 -> 分析报错日志 -> 修正 Payload”的复杂流程。这种能够根据环境反馈动态调整策略的能力，标志着 LLM 正在从辅助工具向“数字安全专家”进化。

三、 技术现状深度剖析与基准评估

1.静态分析能力的虚与实

在静态代码审计场景下，LLM 展现出了发现逻辑漏洞（Logic Bugs）的巨大潜力。例如，在检测“未授权访问”（IDOR）漏洞时，传统静态分析往往因为数据流在语法上是合法的而将其忽略，但 LLM 可以通过分析函数名（如 `getUserInfo`）、变量名（如 `admin_id`）以及上下文逻辑（如未检查 `Session`），推断出业务逻辑上的违规。然而，Tamberg 和 Bahsi（IEEE Access 2024）的大规模基准测试揭示了当前技术的真实水平[3]。结果显示，虽然 LLM 在召回率（Recall）上表现优异，能够发现传统工具漏掉的疑难杂症，但其精确率（Precision）往往较低。这意味着 LLM 目前更适合作为“第二意见”工具，用于查漏补缺，而非完全替代传统方案。此外，研究还发现 LLM 在“语句级定位”上的表现远不如其在“函数级分类”上的表现，这暗示模型可能更多是基于代码的整体风格和局部模式来判断风险，而非进行了精确的数据流全路径分析。

2.动态利用与修复的自动化

在动态分析与漏洞修复方面，LLM 表现出了独特的优势。在针对私有协议或复杂文件格式的测试中，LLM 可以通过阅读 Wireshark 抓包数据或文件规范，推断出协议状态机，并生成覆盖各种状态转换的测试报文。在漏洞修复（APR）领域，对于缓冲区溢出、空指针解引用等常见 CWE 类型，LLM 生成的补丁通过率往往高于传统的基于搜索或约束求解的修复工具，且生成的代码更符合人类开发者的编码习惯。更重要的是，LLM 能够用自然语言解释漏洞成因，这对于安全运营中心（SOC）的分析师来说极具价值，能够大幅降低人工复核的成本。

四、 面临的深层挑战与技术瓶颈

尽管前景广阔，但要将基于 LLM 的系统部署到企业级 DevSecOps 流程中，仍需跨越几座难以逾越的“大山”。

首当其冲的是数据集污染与评估偏差。这是一个在学术界被广泛讨论的危机。许多 LLM 在预训练阶段可能已经“看过”了 GitHub 上的历史漏洞及其修复代码（例如著名的 OpenSSL Heartbleed）。因此，当研究者使用 BigVul、Devign 等经典公开数据集进行测试时，模型的高准确率很可能源于“记忆”而非真正的逻辑推理。2025 年的新趋势是构建基于“零日漏洞”（0-day）或人工合成漏洞的全新基准测试集，初步结果显示，在去除记忆效应后，LLM 的实际性能会有显著下降。

其次是幻觉（Hallucination）与误报难题。LLM 本质上是基于概率预测下一个 Token 的生成模型，而非基于逻辑的验证器。这导致模型经常表现出“过度自信”的倾向，即一本正经地将安全的代码标记为漏洞。例如，它可能会认为一个自定义的加密函数是不安全的，仅仅因为它不符合标准库的命名规范。这种基于“直觉”而非“事实”的误报，是当前阻碍其工业化落地的最大障碍。目前的 Prompt 工程虽然引入了“请引用具体代码行”、“请给出置信度”等约束，但并未从根本上解决这一概率性缺陷。

最后是跨文件（Repository-level）上下文理解的局限。真实世界的软件项目往往包含数千个文件，漏洞的触发通常涉及复杂的跨函数、跨文件调用链。尽管 Gemini 1.5 Pro 等模型已支持百万级 Token 的上下文窗口，但直接将整个代码仓库塞入 Context 中不仅成本高昂，还会引发“迷失中间”（Lost-in-the-Middle）现象，即模型忽略掉中间的关键信息。目前的解决方案主要依赖检索增强生成（RAG）或程序切片技术，试图提取与漏洞相关的代码片段。然而，如果检索算法遗漏了某个关键的全局变量定义或结构体声明，LLM 的推理基础就会崩塌，导致分析结果失效。

五、 结论与未来展望

综上所述，2025 年的基于 LLM 的软件漏洞分析技术正处于从“验证可行性”向“解决工程化难题”跨越的关键爬坡期。LLM 并未宣告传统安全技术的终结，而是为其插上了理解语义的翅膀。在代码审计的可解释性、模糊测试输入的生成质量以及渗透测试的自动化决策方面，LLM 已展现出不可替代的价值。

展望未来，技术的突破点将主要集中在神经符号人工智能（Neuro-symbolic AI）的深度融合上。未来的高可信系统将不再是纯粹的端到端神经网络，而是“大模型 + 形式化求解器”的混合体。LLM 发挥其强大的语义理解与假设生成能力，负责将自然语言或代码转化为逻辑约束（Constraints），然后交由 Z3 等 SMT 求解器或 CodeQL 等静态分析引擎进行数学上的精确验证。这种结合有望同时实现 LLM 的高召回率与传统方法的零误报率。此外，随着多模态大模型的发展，让模型不仅阅读代码文本，还能“看”到程序运行时的内存布局图、控制流图的可视化表示，甚至 IDE 的交互界面，将进一步提升其对复杂软件行为的理解深度。与此同时，出于对代码隐私的保护，基于本地部署的专用小模型（7B-13B 参数量级）配合私有知识库的 RAG 方案，将成为企业内部安全建设的标准配置。

参考文献

- [1] Xia, C. S., Paltenghi, M., Tian, J. L., Pradel, M., & Zhang, L. (2024). **Fuzz4All: Universal Fuzzing with Large Language Models.** *Proceedings of the 46th International Conference on Software Engineering (ICSE 2024).*
- [2] Deng, G., Liu, Y., Mayoral-Vilches, V., et al. (2024). **PentestGPT: Evaluating and Harnessing Large Language Models for Automated Penetration Testing.** *33rd USENIX Security Symposium.*
- [3] Tamberg, K., & Bahsi, H. (2024). **Harnessing Large Language Models for Software Vulnerability Detection: A Comprehensive Benchmarking Study.** *IEEE Access*, Vol. 13.